Interview Model — Detailed Reference

Overview

The interview domain is composed of three database models (`Interview` , `Turn` , `Feedback`) and a set of Pydantic schemas for API contracts. Together they represent the full lifecycle of a mock interview session.

Enums

`InterviewStatus`

Tracks the interview lifecycle. Flow: `PENDING → PROCESSING_CV → READY → IN_PROGRESS → COMPLETED` . `FAILED` can occur from any state.

Value	Description
`pending`	Just created, no processing yet
`processing_cv`	AI is analyzing CV/JD
`ready`	Analysis complete, ready to start
`in_progress`	WebSocket session active
`completed`	Interview finished
`failed`	Processing or interview failed

`InterviewPhase`

Defines the structured progression of questions within a session.

Value	Description
`intro`	Opening question, get to know candidate

Value	Description
`behavioral`	STAR-format situational questions
`technical`	Domain-specific technical questions
`deep_dive`	Follow-up probes on strengths/weaknesses
`closing`	Candidate questions, wrap-up

Models

`Interview`

Table: `interviews`

The main session record. Stores the full context needed to run and resume an interview.

Column	Type	Nullable	Description
`id`	UUID	No	Primary key
`user_id`	UUID (FK → users)	No	Owner of the interview
`status`	String(20)	No	Current `InterviewStatus`
`cv_raw_text`	Text	Yes	Original extracted CV text
`cv_analysis`	JSON	Yes	Structured AI analysis of the CV
`jd_raw_text`	Text	Yes	Original job description text
`jd_analysis`	JSON	Yes	Structured AI analysis of the JD
`interview_config`	JSON	Yes	Phase config, question counts, difficulty
`target_role`	String(255)	Yes	Job title extracted from JD
`target_company`	String(255)	Yes	Company name if specified
`session_state`	JSON	Yes	Snapshot for crash recovery

Column	Type	Nullable	Description
`current_phase`	String(20)	Yes	Active `InterviewPhase`
`current_turn`	Integer	No	Turn counter (0 = not started)
`completed_at`	DateTime	Yes	Timestamp of completion
`total_duration_seconds`	Float	Yes	Total session duration
`created_at`	DateTime	No	Auto-set on creation
`updated_at`	DateTime	No	Auto-updated on change

Indexes

Index	Columns	Notes
`idx_interview_user_status`	`user_id` , `status`	Composite — user's interviews by status
`idx_interview_role`	`target_role`	Role-based queries
`idx_interview_completed`	`completed_at`	Completed interview queries

Relationships

Name	Type	Target	Cascade
`user`	Many-to-one	`User`	—
`turns`	One-to-many	`Turn`	`all, delete-orphan`
`feedback`	One-to-one	`Feedback`	`all, delete-orphan`

Key Properties & Methods

Name	Type	Description
`is_ready_to_start`	property	`status == READY`
`is_in_progress`	property	`status == IN_PROGRESS`
`is_completed`	property	`status == COMPLETED`
`has_cv_analysis`	property	`cv_analysis is not None`
`has_jd_analysis`	property	`jd_analysis is not None`

Name	Type	Description
`get_cv_skills()`	method	Returns `cv_analysis.skills.technical` list
`get_jd_requirements()`	method	Returns `jd_analysis.required_skills` list
`get_skill_gap_analysis()`	method	Returns `{matching_skills, missing_skills, additional_skills}`
`to_dict(include_analysis)`	method	Serializes to dict; optionally includes full analysis

`Turn`

Table: `turns`

One question-answer exchange within an interview. Created per AI question, updated when the user answers and the AI evaluates.

Column	Type	Nullable	Description
`id`	UUID	No	Primary key
`interview_id`	UUID (FK → interviews)	No	Parent interview
`turn_number`	Integer	No	Sequential number (1, 2, 3…)
`phase`	String(20)	No	`InterviewPhase` at time of turn
`ai_question`	Text	No	The AI's question
`user_answer`	Text	Yes	Candidate's answer
`evaluation`	JSON	Yes	AI evaluation (relevance, depth, clarity, etc.)
`duration_seconds`	Float	Yes	Time taken to answer
`difficulty_level`	Float	Yes	Question difficulty (0.0 easy → 1.0 hard)
`created_at`	DateTime	No	Auto-set on creation
`updated_at`	DateTime	No	Auto-updated on change

Indexes

Index	Columns	Notes
`idx_turn_interview_number`	`interview_id` , `turn_number`	Composite — ordered turns per interview
`idx_turn_phase`	`phase`	Phase-based queries
`idx_turn_unique`	`interview_id` , `turn_number`	Unique constraint — one turn number per interview

Key Properties & Methods

Name	Type	Description
`has_answer`	property	`user_answer` is non-empty
`has_evaluation`	property	`evaluation is not None`
`get_evaluation_score(metric)`	method	Returns a specific score from `evaluation` JSON
`get_overall_score()`	method	Average of `relevance` , `depth` , `clarity`
`to_dict(include_evaluation)`	method	Serializes to dict; optionally includes evaluation

Feedback

Table: `feedback`

Post-interview analysis generated by the AI after session completion. One-to-one with `Interview` .

Column	Type	Nullable	Description
`id`	UUID	No	Primary key
`interview_id`	UUID (FK → interviews)	No	Parent interview (unique)
`overall_score`	Float	No	Overall score (0–100)
`summary`	Text	Yes	Brief performance summary

Column	Type	Nullable	Description
`strengths`	JSON (list)	No	Identified strengths with evidence
`weaknesses`	JSON (list)	No	Identified weaknesses with evidence
`suggestions`	JSON (list)	No	Actionable improvement suggestions
`phase_scores`	JSON (dict)	No	Score per `InterviewPhase`
`skill_assessment`	JSON	Yes	Per-skill scores from JD requirements
`detailed_analysis`	Text	Yes	Full narrative feedback
`generation_metadata`	JSON	Yes	AI model, tokens used, etc.
`created_at`	DateTime	No	Auto-set on creation
`updated_at`	DateTime	No	Auto-updated on change

Indexes

Index	Columns
`idx_feedback_score`	`overall_score`
`idx_feedback_created`	`created_at`

Performance Levels (derived from `overall_score`)

Score Range	Level
≥ 90	`excellent`
≥ 80	`good`
≥ 70	`satisfactory`
≥ 60	`needs_improvement`
< 60	`poor`

Key Properties & Methods

Name	Type	Description
`performance_level`	property	Derived label from `overall_score`
`top_strengths`	property	Top 3 strengths sorted by score
`top_weaknesses`	property	Top 3 weaknesses sorted by score (ascending)
`high_priority_suggestions`	property	Suggestions where `priority == "high"`
`get_phase_score(phase)`	method	Score for a specific phase
`get_skill_score(skill)`	method	Score for a specific skill
`to_dict(include_detailed)`	method	Serializes to dict; optionally includes full analysis
`create_from_analysis(...)`	classmethod	Factory method for creating from AI output

Pydantic Schemas

Request Schemas

Schema	Purpose	Key Validation
`CreateInterviewRequest`	Create a new interview	`jd_text` required, 100–10,000 chars
`UpdateInterviewRequest`	Update interview metadata	Optional `target_company`
`InterviewActionRequest`	Trigger actions	`action` must be `start`, `complete`, or `pause`

Response Schemas

Schema	Purpose
`InterviewSummaryResponse`	Lightweight list item
`InterviewDetailResponse`	Full detail with optional analysis
`TurnResponse`	Single turn with optional evaluation

Schema	Purpose
`InterviewListResponse`	Paginated interview list
`InterviewStatsResponse`	Aggregate user statistics
`CVAnalysisResponse`	Structured CV analysis
`JDAnalysisResponse`	Structured JD analysis
`SkillGapAnalysisResponse`	Matching/missing/additional skills
`FileUploadResponse`	File upload result

Utility Functions

Function	Description
`create_interview_summary_response(interview)`	Builds `InterviewSummaryResponse` from ORM model
`create_interview_detail_response(interview, include_analysis)`	Builds `InterviewDetailResponse`, optionally with full CV/JD/gap analysis

Relationships Diagram

```
User (1) ──────────────── (many) Interview
                                  │
                      ┌───────────┤
                      │           │
                (many) Turn     (one) Feedback
```

JSON Column Structures

cv_analysis

```
{
  "candidate_name": "string",
  "years_of_experience": 5,
  "current_role": "string",
  "seniority_level": "mid | senior | junior",
  "skills": {
    "technical": ["Python", "FastAPI"],
    "soft": ["Communication"]
  },
  "experience": [
    {
      "role": "string",
      "company": "string",
      "duration": "string",
      "key_achievements": [],
      "technologies_used": []
    }
  ],
  "education": [],
  "projects": [],
  "certifications": [],
  "notable_points": [],
  "potential_gaps": [],
  "interview_focus_areas": []
}
```

jd_analysis

```
{
  "role_title": "string",
  "company": "string | null",
  "seniority_level": "string",
  "required_skills": [],
  "preferred_skills": [],
  "key_responsibilities": [],
  "technical_requirements": [],
  "interview_focus_areas": [],
  "required_experience": {},
  "company_culture": {}
}
```

evaluation (per Turn)

```
{
  "overall_score": 7.5,
  "relevance": 8.0,
  "depth": 7.0,
  "clarity": 7.5,
  "strengths": [],
  "areas_for_improvement": [],
  "criteria_scores": {
    "technical_knowledge": 7.0,
    "communication_skills": 8.0
  }
}
```

strengths / weaknesses (Feedback)

```
[
  {
    "area": "string",
    "evidence": "string",
    "score": 8.5
  }
]
```

`suggestions` (Feedback)

```
[
  {
    "action": "string",
    "priority": "high | medium | low"
  }
]
```